

8 模板方法模式

封装算法



目前为止我们都在围着封装转；我们封装对象创建、方法调用、复杂接口、鸭子、比萨……接下来呢？我们打算深入到封装算法块，好让子类可以在任何时候，把自己挂接进计算中。我们甚至会在本章学到一个由好莱坞激发的设计原则。我们开始……

是时候来更多的咖啡因了

有些人没有咖啡就活不了；有些人没有茶就活不了。
两者共同的成分是什么？当然是咖啡因！

但还有更多，茶和咖啡的制作方式非常类似。我们
来看看：

星巴克咖啡师傅训练手册

各位师傅！准备星巴克饮料时，请精确地遵循下面的冲泡法：

星巴克咖啡冲泡法

- (1) 把水煮沸。
- (2) 用沸水冲泡咖啡。
- (3) 把咖啡倒进杯子。
- (4) 加糖和奶。

星巴克茶冲泡法

- (1) 把水煮沸。
- (2) 用沸水浸泡茶叶。
- (3) 把茶倒进杯子。
- (4) 加柠檬。

所有冲泡法都是星巴克咖啡公司的商业机密，应严格保密。

咖啡的冲泡法看
起来和茶很像，
不是吗？

快速搞定一些咖啡和茶的类 (用Java)



我们来扮演“代码师傅”，编写一些创建咖啡和茶的代码。

咖啡如下：

这是我们的Coffee类，用来做咖啡。

```
public class Coffee {
```

```
    void prepareRecipe() {
```

```
        boilWater();
```

```
        brewCoffeeGrinds();
```

```
        pourInCup();
```

```
        addSugarAndMilk();
```

```
    }
```

```
    public void boilWater() {
```

```
        System.out.println("Boiling water");
```

```
    }
```

```
    public void brewCoffeeGrinds() {
```

```
        System.out.println("Dripping Coffee through filter");
```

```
    }
```

```
    public void pourInCup() {
```

```
        System.out.println("Pouring into cup");
```

```
    }
```

```
    public void addSugarAndMilk() {
```

```
        System.out.println("Adding Sugar and Milk");
```

```
    }
```

```
}
```

这是我们的咖啡冲泡法，直接取自训练手册。

每个步骤都实现为分离的方法。

这些方法的每一个都实现算法的一个步骤：煮沸水、冲泡咖啡、倒咖啡入杯、加糖和奶。

现在是茶……



```
public class Tea {

    void prepareRecipe() {
        boilWater();
        steepTeaBag();
        pourInCup();
        addLemon();
    }

    public void boilWater() {
        System.out.println("Boiling water");
    }

    public void steepTeaBag() {
        System.out.println("Steeping the tea");
    }

    public void addLemon() {
        System.out.println("Adding Lemon");
    }

    public void pourInCup() {
        System.out.println("Pouring into cup");
    }
}
```

这看起来和刚刚实现的*Coffee*很像。第二个和第四个步骤不一样，但基本上是相同的冲泡法。

这两个方法是泡茶专有的。

注意，这两个方法和咖啡的完全一样！因此这里肯定有一些重复的代码。

有代码重复，是一个我们需要清理设计的迹象。在这里，看起来我们应该把共性抽象到一个基类，因为咖啡和茶如此相似。



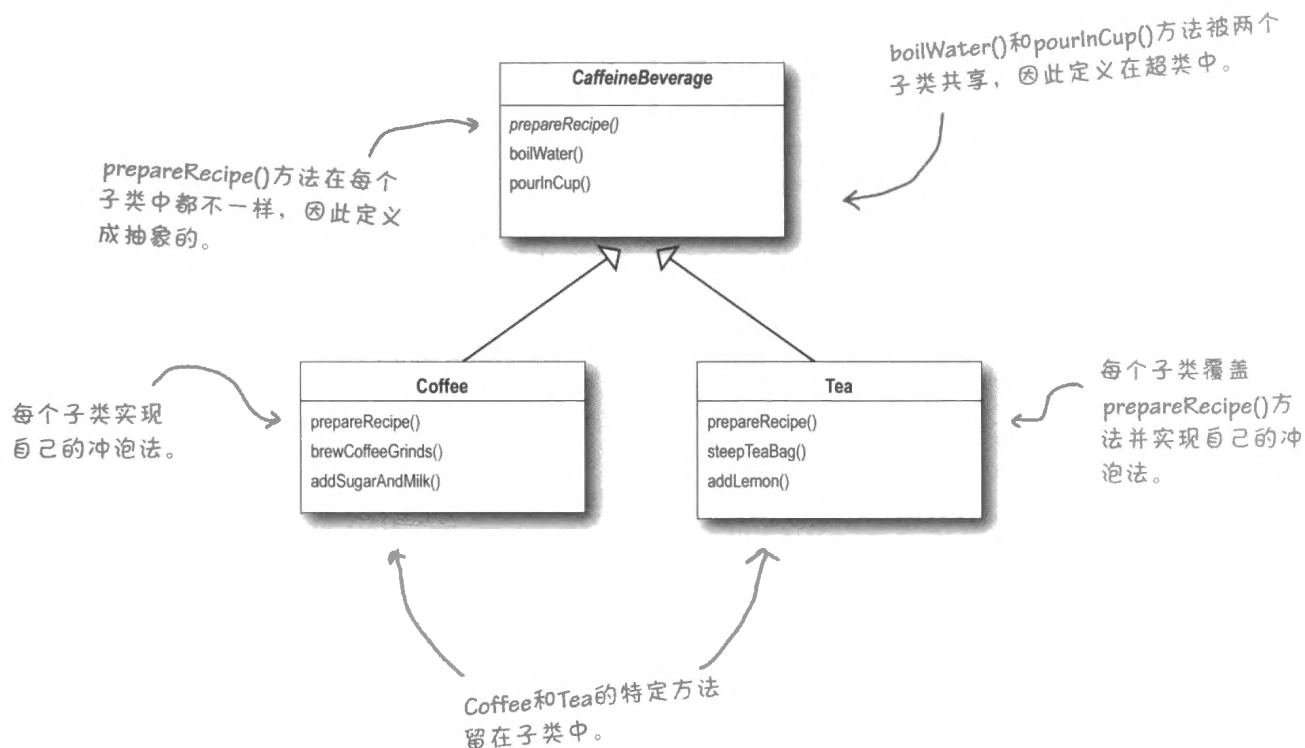


设计谜题

你已经看到Coffee和Tea类有相当部分代码是重复的。再看一下Coffee和Tea类，画一张类图，展示你如何重新设计类以移除冗余。

我们来抽象Coffee和Tea

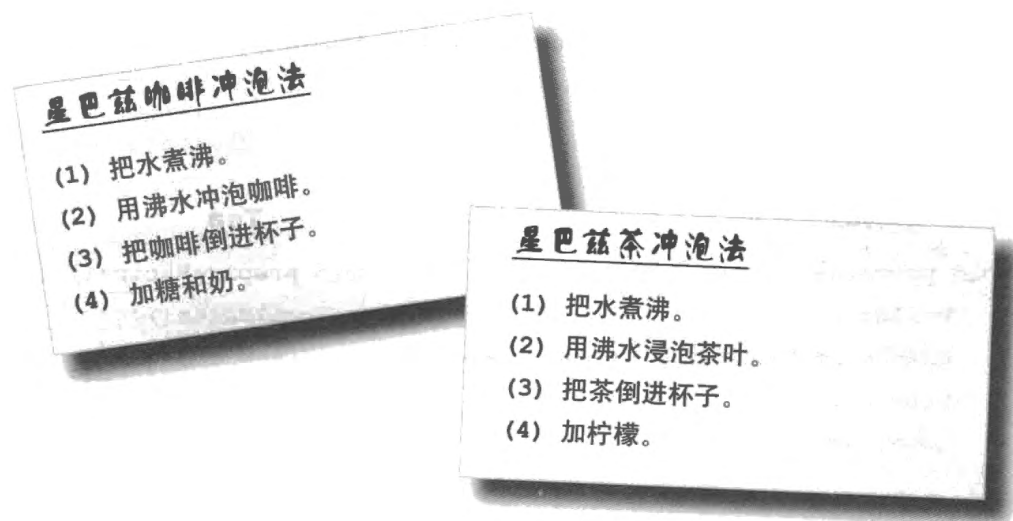
看起来，在编写Coffee和Tea类时，我们经历了一次相当直接的设计练习。你的第一刀可能看起来像这样：



我们的重新设计是不是很棒？嗯，再看一眼，我们是不是忽略了某些共性？Coffee和Tea还有什么相似的地方？

进一步设计……

Coffee和Tea还有什么共同点？我们从冲泡法开始。



注意，两份冲泡法都遵循相同的算法：

❶ 把水煮沸。

❷ 用沸水提取咖啡或茶。

❸ 把得到的饮料倒进杯子。

❹ 给饮料添加适合的调料。

这两个没有被抽象，
但它们是一样的，只
是应用在不同的饮料。

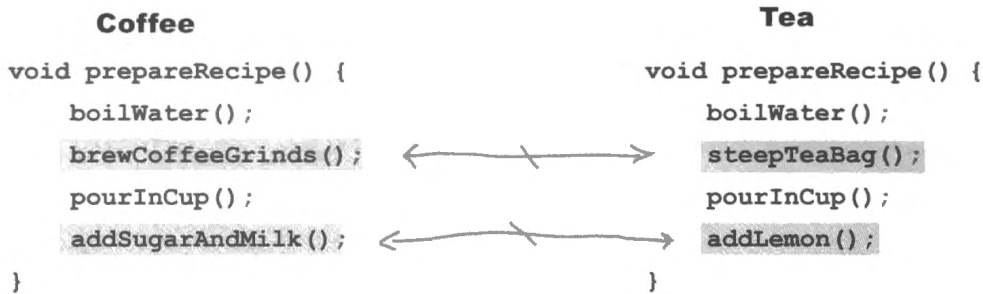
这两个已经被抽象
到基类中。

那么，我们能想办法也抽象prepareRecipe()吗？是的，我们来看看怎么做……

抽象prepareRecipe()

我们逐步从每个子类（就是Coffee和Tea类）抽象prepareRecipe()……

- 1 第一个问题是，Coffee使用brewCoffeeGrinds()和addSugarAndMilk()方法，而Tea使用steepTeaBag()和addLemon()方法。



我们来全面考虑一下：浸泡（steeping）和冲泡（brewing）差异没那么大。它们相当类似。因此，我们来想一个新的方法名，比如说brew()，不管是冲泡咖啡还是浸泡茶叶，我们将使用同样的名称。

同样，添加糖奶和添加柠檬也相当类似：都是给饮料添加调料。我们来想一个新的方法名，addCondiments()。因此，我们的新prepareRecipe()方法看起来像这样：

```

void prepareRecipe() {
    boilWater();
    brew();
    pourInCup();
    addCondiments();
}
```

- 2 现在我们有了新的prepareRecipe()方法，但我们需要把它融合进代码。为此，我们先从CaffeineBeverage（咖啡因饮料）超类开始：

（代码在下页。）

如同类设计中, CaffeineBeverage
是抽象的。

```
public abstract class CaffeineBeverage {

    final void prepareRecipe() {
        boilWater();
        brew();
        pourInCup();
        addCondiments();
    }

    abstract void brew();

    abstract void addCondiments();

    void boilWater() {
        System.out.println("Boiling water");
    }

    void pourInCup() {
        System.out.println("Pouring into cup");
    }

}
```

现在, 同一个prepareRecipe()方法会被用于处理Tea和Coffee, prepareRecipe()声明为final, 因为我们不希望子类覆盖这个方法并改变冲泡法! 我们将步骤2和4泛化为brew()和addCondiments()。

因为Coffee和Tea处理这些方法的做法不同, 因此这两个方法必须声明为抽象。让子类操心具体细节。

记住, 我们将这些移进CaffeineBeverage类(回到我们的类图)。

- 3 最后, 我们需要处理Coffee和Tea类。它们现在依靠CaffeineBeverage来处理冲泡法, 因此它们只需要处理冲泡和添加调料的部分:

```
public class Tea extends CaffeineBeverage {
    public void brew() {
        System.out.println("Steeping the tea");
    }
    public void addCondiments() {
        System.out.println("Adding Lemon");
    }
}
```

正如设计中, Tea和Coffee现在扩展CaffeineBeverage。

Tea需要定义brew()和addCondiments(), 这两个抽象方法来自CaffeineBeverage。

```
public class Coffee extends CaffeineBeverage {
    public void brew() {
        System.out.println("Dripping Coffee through filter");
    }
    public void addCondiments() {
        System.out.println("Adding Sugar and Milk");
    }
}
```

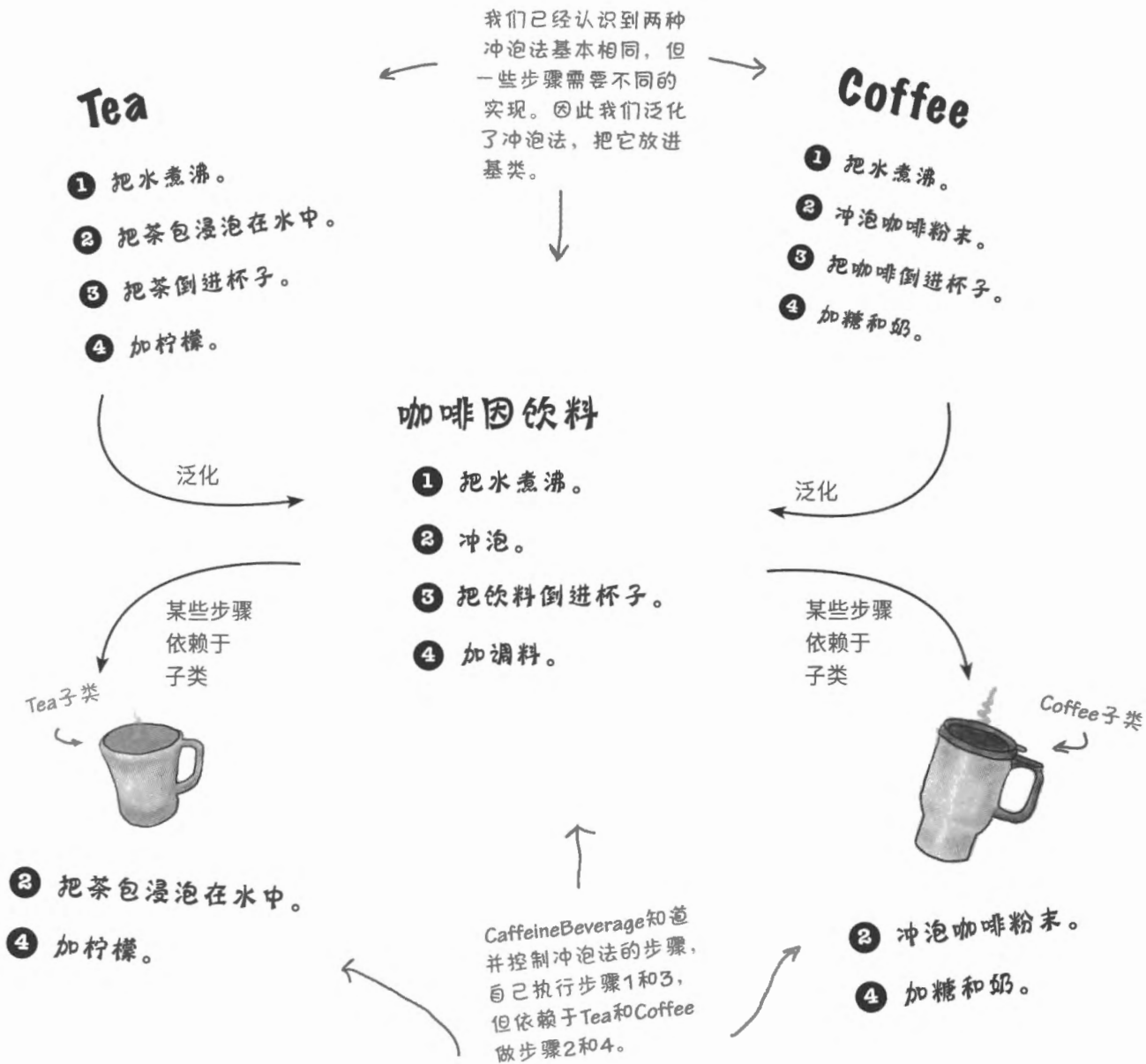
Coffee也一样, 除了Coffee处理的是咖啡, 用糖和奶取代了茶包和柠檬。



削尖你的铅笔

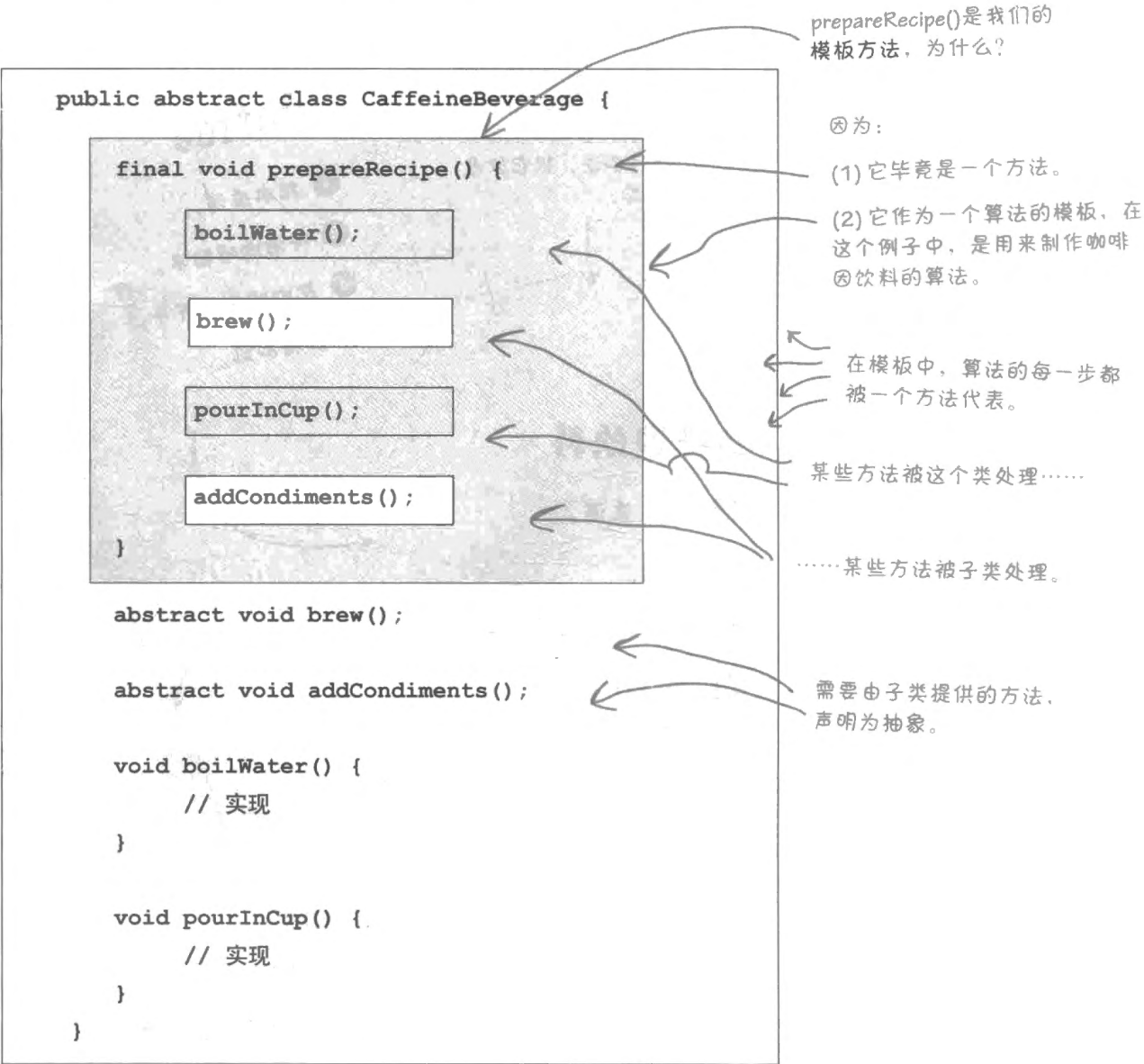
既然我们已经把prepareRecipe()的实现移进CaffeineBeverage类，那就画一张新的类图：

我们做了什么？



认识模板方法

我们基本上就是实现了模板方法模式。模板方法模式是什么？我们来看看CaffeineBeverage类的结构，它包含实际的“模板方法”：



模板方法定义一个算法的步骤，允许子类提供一个或多个步骤的实现。

我们来泡茶……

我们来一步步泡茶并追踪模板方法如何运作。你会看到模板方法控制算法。在算法的某一点，它让子类提供步骤的实现……

幕后花絮



- 1 OK，首先我们需要一个Tea对象……

```
Tea myTea = new Tea();
```

- 2 然后我们调用模板方法：

```
myTea.prepareRecipe();
```

模板方法遵循制作咖啡因饮料的算法……

```
boilWater();
brew();
pourInCup();
addCondiments();
```

prepareRecipe()方法控制算法。没有人能改变这一点，它依赖子类来提供某些或所有步骤地实现。

- 3 首先我们烧水：

```
boilWater();
```

这发生在CaffeineBeverage中。

- 4 接下来，我们需要泡茶，只有子类知道如何做：

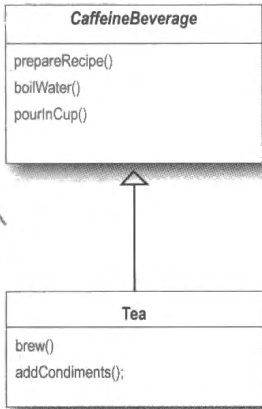
```
brew();
```

- 5 现在我们把茶倒进杯子；这一点所有饮料都一样，因此它发生在CaffeineBeverage中：

```
pourInCup();
```

- 6 最后，我们加调料，这对每种饮料来说是特定的，因此子类实现这一点：

```
addCondiments();
```



模板方法带给我们什么？



动力不足的Tea & Coffee 实现

Coffee和Tea主导这场秀，它们控制算法。

代码在Coffee和Tea之间重复。

算法的代码变更需要打开子类，并改变多个地方。

类的组织结构使得添加一个新的咖啡因饮料需要很多工作。

算法以及如何实现算法的知识被分布在很多类中。



由模板方法赋能的新的、 时髦的CaffeineBeverage

CaffeineBeverage类主导这场秀，它有算法并且保护算法。

CaffeineBeverage类在子类之间的复用最大化。

算法只存在一个地方，只需要修改那里的代码。

模板方法模式提供一个框架，可以让其他咖啡因饮料插进来。新的咖啡因饮料只需要实现两个方法。

CaffeineBeverage类集中了关于算法的知识，依赖子类提供完整的实现。

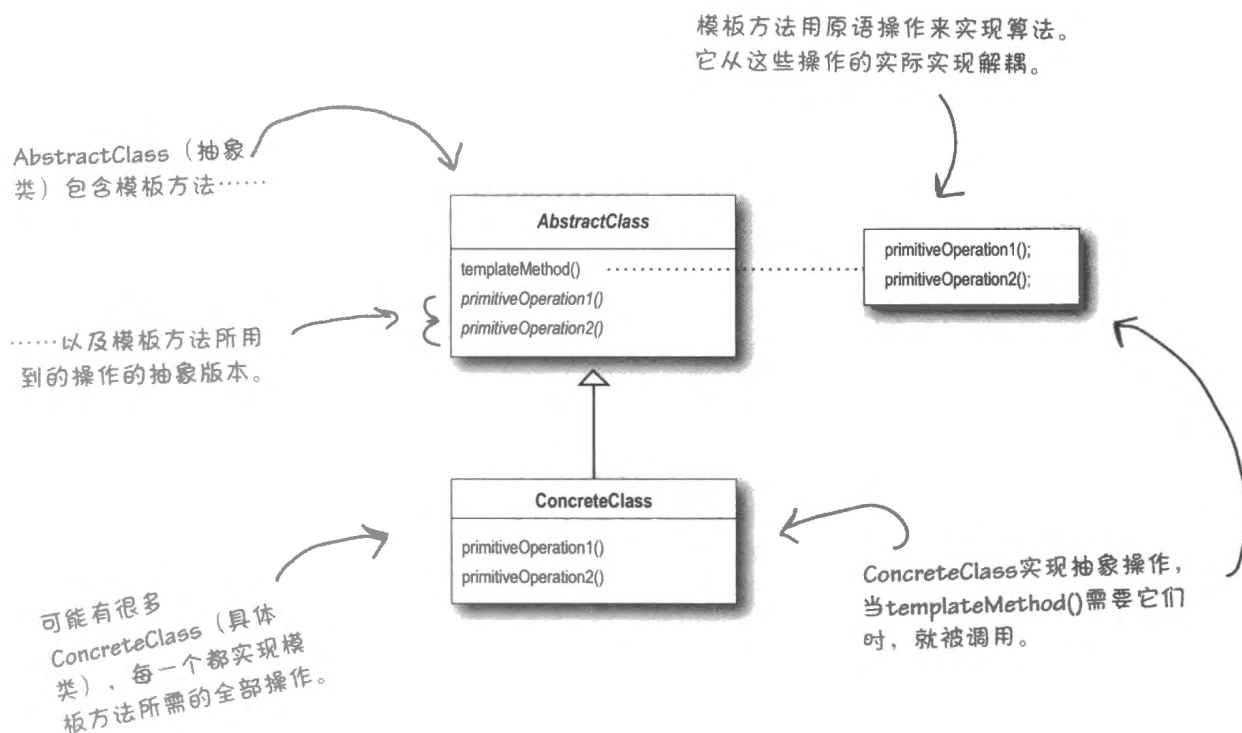
定义模板方法模式

你已经看到，Tea和Coffee的例子中模板方法模式是怎样工作的。现在，来看看官方定义并弄清所有细节：

模板方法模式 在一个方法中定义一个算法的骨架，而把一些步骤延迟到子类。模板方法使得子类可以在不改变算法结构的情况下，重新定义算法中的某些步骤。

这个模式是用来创建一个算法的模板。什么是模板？正如你已经看到的，它就是一个方法，更具体地说，它是一个把算法定义为一组步骤的方法。这些步骤中的一个或多个被定义为抽象的，由子类实现。这保证了算法的结构保持不变，而子类提供某些部分的实现。

我们来看看类图：





Code up Close

我们来看得更仔细一点，看看AbstractClass如何定义的，包括模板方法和原语操作。

这里是抽象类：它被声明为abstract，意味着由提供操作实现的类所继承。

```
abstract class AbstractClass {
```

```
    final void templateMethod() {  
        primitiveOperation1();  
        primitiveOperation2();  
        concreteOperation();  
    }
```

这是模板方法。它被声明为final，以防止子类改变算法步骤的顺序。

模板方法定义步骤的顺序，每一步由一个方法代表。

```
    abstract void primitiveOperation1();
```

```
    abstract void primitiveOperation2();
```

在这个例子中，有两个原语操作，具体子类必须实现它们。

```
    void concreteOperation() {  
        // 在这里实现  
    }
```

我们也有具体操作定义在抽象类。这个操作可以被子类覆盖，或者我们可以通过声明concreteOperation()为final来防止覆盖。下面还会说到更多……



Code way Up Close

现在我们打算再靠近一点看，看看抽象类中可以有哪些类型的方法：

我们修改 `templateMethod()`
为包括一个新的方法调用。

```
abstract class AbstractClass {

    final void templateMethod() {
        primitiveOperation1();
        primitiveOperation2();
        concreteOperation();
        hook();
    }

    abstract void primitiveOperation1();

    abstract void primitiveOperation2();

    final void concreteOperation() {
        // 在这里实现
    }

    void hook() {}

}
```

我们依然有我们的原语操作方法，这是抽象的，由具体子类实现。

一个定义在抽象类中的具体操作。它声明为 `final`，这样一来子类就不能覆盖它。它可以直接用在模板方法中，或者被子类使用。

一个具体方法，但它什么都不做！

我们也可以有缺省不做事的具体方法，我们称为“钩子”。子类可以视情况覆盖，但不一定。在下一页，我们会看到钩子如何使用。

挂钩模板方法……

钩子是一个声明在抽象类中的方法，但它只给出空的或缺省的实现。这给了子类在变化点“挂钩进”算法的能力，如果需要，子类也有权忽略钩子。

钩子有几种用途。现在我们来看一个，其他的稍后再看：

有了钩子，我可以覆盖或不覆盖方法。这是我的选择。如果我不覆盖，抽象类提供缺省的实现。



```
public abstract class CaffeineBeverageWithHook {
```

```
    final void prepareRecipe() {
```

```
        boilWater();
```

```
        brew();
```

```
        pourInCup();
```

```
        if (customerWantsCondiments()) {
            addCondiments();
        }
    }
```

我们添加一条小小的条件语句。条件是否成立，由一个具体方法 `customerWantsCondiments()` 决定。顾客想要调料时，才调用 `addCondiments()`。

```
    abstract void brew();
```

```
    abstract void addCondiments();
```

```
    void boilWater() {
```

```
        System.out.println("Boiling water");
```

```
    }
```

```
    void pourInCup() {
```

```
        System.out.println("Pouring into cup");
```

```
    }
```

```
    boolean customerWantsCondiments() {
```

```
        return true;
```

```
    }
```

```
}
```

在这里，我们定义了一个方法，缺省实现（通常）为空。这个方法只返回 `true`，不做别的事。

这是一个钩子，因为子类可以覆盖这个方法，但不一定要这么做。

使用钩子

为了使用钩子，我们在子类覆盖它。在这里，钩子控制CaffeineBeverage类是否执行某部分算法——即，是否给饮料加调料。

我们怎么知道顾客是否要调料？问呗！

```
public class CoffeeWithHook extends CaffeineBeverageWithHook {

    public void brew() {
        System.out.println("Dripping Coffee through filter");
    }

    public void addCondiments() {
        System.out.println("Adding Sugar and Milk");
    }

    public boolean customerWantsCondiments() {

        String answer = getUserInput();

        if (answer.toLowerCase().startsWith("y")) {
            return true;
        } else {
            return false;
        }
    }

    private String getUserInput() {
        String answer = null;

        System.out.print("Would you like milk and sugar with your coffee (y/n)? ");

        BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
        try {
            answer = in.readLine();
        } catch (IOException ioe) {
            System.err.println("IO error trying to read your answer");
        }
        if (answer == null) {
            return "no";
        }
        return answer;
    }
}
```

在这里，你覆盖了钩子并提供自己的功能。

取得关于调料决策的用户输入，根据输入返回true或false。

这段代码询问用户是否想要奶和糖，从命令行获得用户输入。

我们来运行测试程序

OK, 水正在沸腾……下面是创建热茶和热咖啡的测试代码。

```
public class BeverageTestDrive {
    public static void main(String[] args) {

        TeaWithHook teaHook = new TeaWithHook();
        CoffeeWithHook coffeeHook = new CoffeeWithHook();

        System.out.println("\nMaking tea...");
        teaHook.prepareRecipe();

        System.out.println("\nMaking coffee...");
        coffeeHook.prepareRecipe();
    }
}
```

← 创建茶。

← 创建咖啡。

← 两者都是调用 prepareRecipe()!

我们来运行一下……

```
File Edit Window Help send-more-honesttea
%java BeverageTestDrive
Making tea...
Boiling water
Steeping the tea
Pouring into cup
Would you like lemon with your tea (y/n)? y
Adding Lemon

Making coffee...
Boiling water
Dripping Coffee through filter
Pouring into cup
Would you like milk and sugar with your coffee (y/n)? n
%
```

一杯热腾腾的茶, 是的, 当然要柠檬!

←

一杯温度正好的咖啡, 但不要加让人长胖的调料。

←



你知道吗？我们同意你的看法。但是在这样想之前，你不得不承认，对于钩子如何用作条件来控制抽象类中的算法流，它是一个相当酷的例子。对吧？

我们肯定，在你自己的代码中，你可以想到很多其他真实的场景可以使用模板方法和钩子。

没有蠢问题

问：我在创建模板方法时，怎么知道何时使用抽象方法，何时使用钩子？

答：在子类必须提供方法实现或者算法中的步骤时，使用抽象方法。当算法的某部分可选时，使用钩子。用钩子时，子类可以选择实现钩子，但不一定。

问：钩子的真正使用目的是什么？

答：钩子有一些用法。正如我们刚说过的，钩子提供一种让子类实现算法可选部分的方式，或者如果该部分对子类的实现不重要，可以忽略。另一种用法是给予类一个机会，对模板方法中某些即将发生的（或刚刚发生的）步骤作出反应。例如，钩子方法justReorderedList()允许子类在内部列表重新排序后执行某些活动（例如刷新屏幕信息）。正如你已经看到的，钩子也可以给予类提供为抽象类做决定的能力。

问：子类是否不得不实现AbstractClass的所有抽象方法？

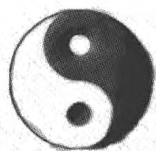
答：是的，每个具体子类定义一整套抽象方法，并为模板方法算法的未定义步骤提供完整的实现。

问：看起来我应该把我的抽象方法保持在小的数量，否则，在子类中实现它们工作量很大。

答：当你编写模板方法时，记住这一点是好事。有时你可以通过不把算法的步骤切得太小来避免这个问题，但这显然是一个折中：步骤越少，弹性越小。也要记住，有些步骤是可选的，因此你可以实现为钩子，而不是抽象方法，从而减轻子类的负担。

好莱坞原则

我们得到了另一个设计原则，称为好莱坞原则：



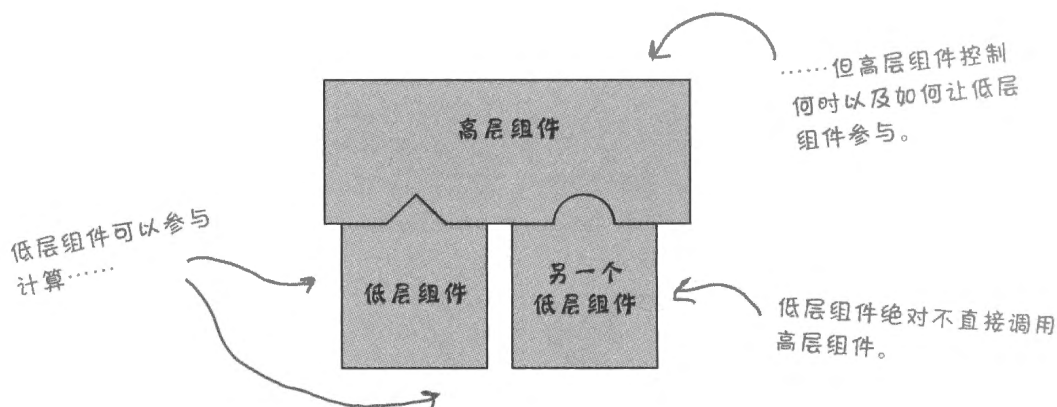
好莱坞原则

不要打电话给（调用）我们，我们会打电话给（调用）你。

很容易记住，对吧？但这和OO设计有什么关系？

好莱坞原则给我们一种防止“依赖腐烂”的方法。“依赖腐烂”的例子如高层组件依赖于低层组件，低层组件又依赖于高层组件，而高层组件又依赖于横向组件，横向组件又依赖于低层组件等。当腐烂蔓延时，没有人能轻易理解系统是怎么设计的。

有了好莱坞原则，我们允许低层组件把自己挂钩进系统，但由高层组件决定何时需要它们，以及怎样需要它们。换句话说，高层组件对低层组件的态度是“不要打电话给我，我们会打电话给你。”



我之前说的听到了吧。我再说一遍：不要打电话给我，我会打电话给你！

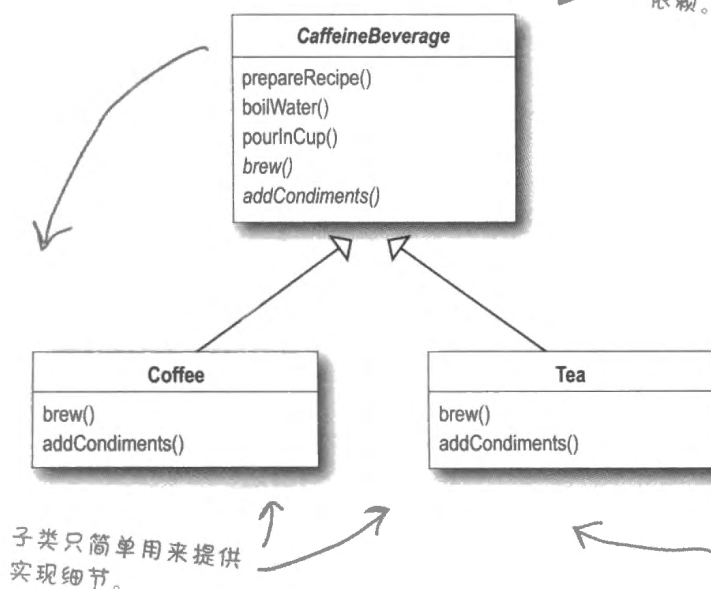


好莱坞原则和模板方法

好莱坞原则和模板方法模式之间的关系可能还是相当明显的：当我们用模板方法模式设计时，我们在告诉子类，“不要打电话给我，我们会打电话给你。” 怎么做？我们再看看CaffeineBeverage的设计：

CaffeineBeverage是我们的高层组件。它控制冲泡法的算法，只有在需要子类实现某个方法时，才调用子类。

饮料的客户依赖CaffeineBeverage抽象，而不依赖具体的Tea或Coffee，这减少了整个系统中的依赖。



还有哪些其他模式采用了好莱坞原则？

工厂方法和观察者，还有其他吗？

没有蠢问题

问：好莱坞原则和前几章所学的依赖倒置原则有什么关系？

答：依赖倒置原则教会我们，避免使用具体类，尽可能利用抽象。好莱坞原则是建造框架或组件的技巧，以便低层组件可以挂钩进计算中，而不会产生低层组件和高层组件之间的依赖。因此，两者都有解耦的目的，但是，在如何避免设计中的依赖上，依赖倒置原则的陈述要强烈和通用得多。

好莱坞原则教给我们一个创建设计的技巧，允许低层结构相互操作，但又防止其他类变得太过依赖它们。

问：低层组件是否不允许调用高层组件的方法？

答：其实不是。事实上，低层组件经常会最终调用在其继承层次之上定义的方法，但我们要避免创建低层组件和高层组件之间显式的环状依赖。

WHO DOES WHAT?

匹配每个模式及其描述：

模式	描述
模板方法	封装可以互换的行为，并使用委托来决定使用哪个行为。
策略	子类决定如何实现算法中的步骤。
工厂方法	子类决定哪个具体类实例化。

荒野中的模板方法

模板方法模式是一个很常见的模式，你会在荒野中发现大量的这个模式。不过，你还得有一双锐利的眼睛，因为有很多模板方法的实现看起来并不十分像教科书里的设计。

这个模式亮相如此经常，是因为对于创建框架来说，它是很棒的设计工具。框架控制该做什么事情，但让你(使用框架的人)指定框架算法中每个步骤实际发生的细节。

让我们在荒野(OK，说的是Java API)中狩猎模板方法的使用吧……



在训练中，我们研究了经典的模式。
但是，当我们来到外面的现实世界，
必须学会辨识周围的模式。我们也必须
学会辨识模式的变体，因为在现实
世界中，一个正方形的洞并不总是纯
正的正方形。



用模板方法排序

我们经常需要用数组做什么事情？排序！

认识到这一点，Java数组类的设计师给我们提供了一个好用的模板方法来排序。我们看看这个方法怎么操作：



为了更容易解释，我们稍微削减了代码。如果你想看完整的代码，就去弄Java源码来看吧……

实际上，这里我们有两个方法，它们一起提供排序功能。

第一个方法，`sort()`，只是一个辅助（helper）方法。它创建一个数组的拷贝，并把它作为目标数组传给`mergeSort()`方法。它也传入数组的长度，并告知从第一个元素开始排序。

```
public static void sort(Object[] a) {  
    Object aux[] = (Object[])a.clone();  
    mergeSort(aux, a, 0, a.length, 0);  
}
```

`mergeSort()`方法包含排序算法，依赖于`compareTo()`方法的实现来完成算法。如果你对排序如何发生的细节感兴趣，去查看源代码。

```
private static void mergeSort(Object src[], Object dest[],  
    int low, int high, int off)  
{  
    // 许多其他代码在这里  
    for (int i=low; i<high; i++){  
        for (int j=i; j>low &&  
            ((Comparable)dest[j-1]).compareTo((Comparable)dest[j])>0; j--)  
        {  
            swap(dest, j, j-1);  
        }  
    }  
    // 许多其他代码在这里  
}
```

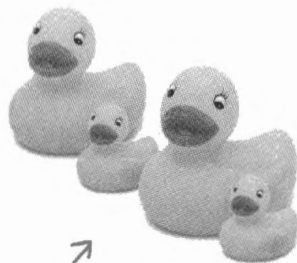
把这个想成模板方法。

我们需要实现`compareTo()`方法，以“填充”模板方法。

这是一个具体方法，已经在数组类中定义。

有一些鸭子要排序……

假设你有一个要排序的鸭子数组，你怎么排序？嗯，数组的`sort()`模板方法给了我们算法，但你需要告诉它如何比较鸭子，做法是：实现`compareTo()`方法……有道理吗？



有一个鸭子数组需要排序。

不，没有道理。
我们不是应该子类化某些东西吗？
我认为这是模板方法的要点。数组不会子类化任何东西，因此我不理解我们如何使用`sort()`。

很好的观点。事情是这样的：`sort()`的设计师希望这个方法能用在所有的数组，因此他们不得不把`sort()`变成一个静态方法，以便可以到处使用。但没关系，因为它工作起来就像它是在超类中。现在，还有一个细节：因为`sort()`不是真正定义在超类中，`sort()`方法需要知道你实现了`compareTo()`方法，否则就没有完成排序算法所需要的片段。

为此，设计师用了`Comparable`接口。你要做的就是实现这个接口，该接口的方法就一个（想不到吧）：`compareTo()`。

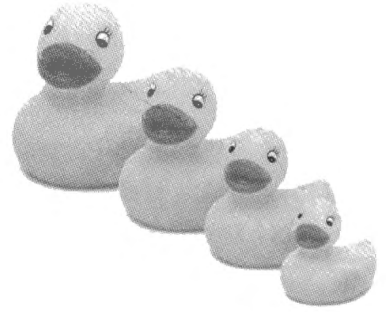
`compareTo()`是什么？

`compareTo()`方法比较两个对象并返回一个是小于、大于还是等于另一个。`sort()`用这个作为比较数组里的对象的基础。

我比你大吗？

我不知道。得
`compareTo()`告诉我们。

比较鸭子和鸭子



OK, 因此你知道如果要排序鸭子, 你不得不实现这个compareTo()方法。为了做到这一点, 你要给数组类它所需要的, 以完成算法并排序鸭子。

鸭子的实现如下:

记住, 我们需要实现Comparable接口,
因为我们不能真的子类化。

```
public class Duck implements Comparable<Duck> {
```

```
    String name;
```

```
    int weight;
```

Duck有名字和体重。

```
    public Duck(String name, int weight) {
```

```
        this.name = name;
```

```
        this.weight = weight;
```

```
    }
```

```
    public String toString() {
```

```
        return name + " weighs " + weight;
```

```
    }
```

我们保持这里简单: Duck所做的只是
打印其名字和体重!

OK, 这是sort()所需要的……

```
    public int compareTo(Duck otherDuck) {
```

compareTo()需要另一只Duck来和这只Duck比较。

```
        if (this.weight < otherDuck.weight) {
```

```
            return -1;
```

```
        } else if (this.weight == otherDuck.weight) {
```

```
            return 0;
```

```
        } else { // this.weight > otherDuck.weight
```

```
            return 1;
```

```
        }
```

```
    }
```

```
}
```

在这里, 我们指定鸭子如何比较。
如果这只鸭子比另一只鸭子轻, 就
返回-1; 如果相等, 就返回0; 如
果这只鸭子更重, 就返回1。

我们来排序一些鸭子

以下是排序鸭子的测试程序……

```
public class DuckSortTestDrive {
```

```
    public static void main(String[] args) {
```

```
        Duck[] ducks = {
```

```
            new Duck("Daffy", 8),
```

```
            new Duck("Dewey", 2),
```

```
            new Duck("Howard", 7),
```

```
            new Duck("Louie", 2),
```

```
            new Duck("Donald", 10),
```

```
            new Duck("Huey", 2)
```

```
        };
```

```
        System.out.println("Before sorting:");
```

```
        display(ducks);
```

```
        Arrays.sort(ducks);
```

```
        System.out.println("\nAfter sorting:");
```

```
        display(ducks);
```

```
    }
```

```
    public static void display(Duck[] ducks) {
```

```
        for (Duck d : ducks) {
```

```
            System.out.println(d);
```

```
        }
```

```
    }
```

```
}
```

← 我们需要一个Duck数组；
这些看起来不错。

← 打印它们，看看它们的名字
和体重。

← 排序时间到！

← 我们（再次）打印它们，看看
它们的名字和体重。

注意，我们调用
Arrays的静态方
法sort()，并传入
ducks。

让排序开始！

File Edit Window Help DonaldNeedsToGoOnADiet

```
%java DuckSortTestDrive
```

```
Before sorting:
```

```
Daffy weighs 8
```

```
Dewey weighs 2
```

```
Howard weighs 7
```

```
Louie weighs 2
```

```
Donald weighs 10
```

```
Huey weighs 2
```

未排序的鸭子

```
After sorting:
```

```
Dewey weighs 2
```

```
Louie weighs 2
```

```
Huey weighs 2
```

```
Howard weighs 7
```

```
Daffy weighs 8
```

```
Donald weighs 10
```

```
%
```

排序后的鸭子

鸭子排序机的制作



我们来跟踪Arrays的sort()模板方法如何运作。我们会看到，模板方法如何控制算法，以及在算法中的某些点上，它如何请求鸭子提供步骤的实现……

- 1 首先我们需要一个鸭子数组。

```
Duck[] ducks = {new Duck("Daffy", 8), ... };
```

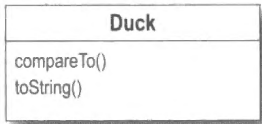
- 2 然后我们调用Arrays类中的sort()模板方法并把ducks传入：

```
Arrays.sort(ducks);
```

sort()方法（及其辅助方法mergeSort()）控制排序过程。

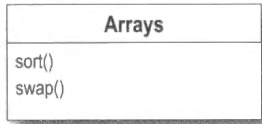
- 3 为了排序一个数组，你需要逐一比较两个项，直到整个数组都排序完毕。

比较两只鸭子时，sort()方法依赖于Duck的compareTo()方法，以知道如何做。compareTo() 方法在第一只鸭子上被调用，并传入要比较的鸭子：



- 4 如果鸭子的次序不对，就用Arrays的具体swap()方法把两者对调：

```
swap()
```



没有继承，不像典型的模板方法。

- 5 sort()方法继续比较并对调鸭子，直到数组处在正确的次序！

```
for (int i=low; i<high; i++){
    ... compareTo() ...
    ... swap() ...
}
```

sort()方法控制算法，没有类可以改变这一点。sort() 依赖一个Comparable类来提供compareTo()的实现。

没有蠢问题

问：这真的是模板方法模式吗，还是你用力过猛了？

答：该模式要求实现一个算法并让子类提供步骤的实现，Arrays的sort()明显没有这样做！但正如我们知道的，荒野中的模式不总是和教科书中的模式一样。它们不得不修正以满足上下文和实现约束。

Arrays的sort()方法的设计师有一些约束。一般来说，你不能子类化一个Java数组，另外他们需要让排序能用于所有数组（每个数组是一个不同的类）。因此他们定义一个静态方法并延迟算法的比较部分到被排序的项。

因此，虽然它不是教科书上的模板方法，但实现依然遵循模板方法模式的精神。还有，由于不再需要子类化Arrays以使用这个算法，在某种形式上，使得排序更有弹性和实用。

问：这个排序的实现看起来确实更像策略模式，而不是模板方法模式。为什么我们把它看成模板方法？

答：你这样想可能是因为策略模式使用对象组合。在某种程度上，你是对的，我们使用Arrays对象来排序数组，因此很像策略。但记住，策略中，你所组合的类实现整个算法。Arrays为sort()实现的算法并不完整，它需要一个类填补遗失的compareTo()方法。因此，它更像模板方法。

问：在JavaAPI中，还有模板方法的其他例子吗？

答：是的，你会在一些地方发现它们。例如，java.io的InputStream有一个read()方法，子类必须实现，并由模板方法 read(byte b[], int off, int len)使用。



我们知道，组合优于继承，对吧？嗯，sort()模板方法的实现决定不使用继承，而是实现成一个静态方法，在运行时和Comparable组合。这样做更好还是更糟？你怎么处理这个问题？是Java的数组让这变得特别棘手吗？



想想看另一个模式，它是模板方法的一个特例。在这个特例中，原语操作用来创建并返回对象，这是什么模式？

随Frame摇摆

接下来的模板方法狩猎中……注意摇动JFrames!



如果你没有接触过JFrame，那稍微介绍一下。它是最基本的Swing容器，继承一个paint()方法。缺省情况下，paint()不做事情，因为它是一个钩子！通过覆盖paint()，你可以把自己的代码插入JFrame的算法中，显示其屏幕区域，把自己的图形输出和JFrame结合。以下是一个使用JFrame覆盖paint()钩子方法的极其简单的例子：

```
public class MyFrame extends JFrame {

    public MyFrame(String title) {
        super(title);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        this.setSize(300,300);
        this.setVisible(true);
    }

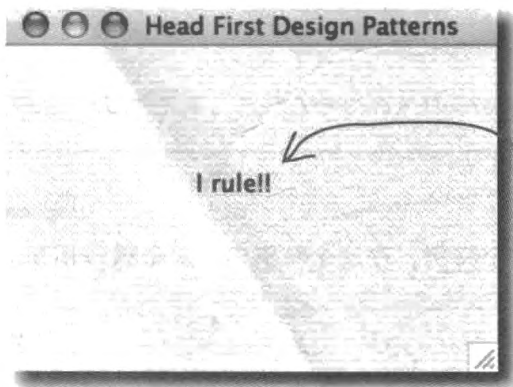
    public void paint(Graphics graphics) {
        super.paint(graphics);
        String msg = "I rule!!";
        graphics.drawString(msg, 100, 100);
    }

    public static void main(String[] args) {
        MyFrame myFrame = new MyFrame("Head First Design Patterns");
    }
}
```

我们扩展JFrame，它包含一个update()方法。这个方法控制更新屏幕的算法。我们可以通过覆盖paint()钩子方法，和这个算法挂上钩。

不要看幕后！这里只是
一些实例化……

JFrame的更新算法称为paint()。缺省情况下，paint()不画任何东西……它是一个钩子。我们覆盖paint()，告诉JFrame在窗口上画出一条消息。



这是一条画在frame中的消息，
因为我们挂钩进paint()方法。

用AbstractList定制列表

我们狩猎的最后一站：AbstractList。

Java中的列表集合，像ArrayList和LinkedList，扩展了AbstractList类，该类为列表行为提供一些基本的实现。如果你要创建自己的定制列表（比如，一个只包含String的列表），可以通过扩展AbstractList，免费获得基本的列表行为，而达到目的。

AbstractList有一个模板方法subList()，这个方法依赖于两个抽象方法get()和size()。因此，当你扩展AbstractList来创建自己定制列表时，你要提供这些方法的实现。

以下是一个定制列表的实现，该列表只包含String对象，并在底层实现使用数组：

```
public class MyStringList extends AbstractList<String> {
    private String[] myList;
    MyStringList(String[] strings) {
        myList = strings;
    }
    public String get(int index) {
        return myList[index];
    }
    public int size() {
        return myList.length;
    }
    public String set(int index, String item) {
        String oldString = myList[index];
        myList[index] = item;
        return oldString;
    }
}
```

我们通过扩展AbstractList创建一个定制列表。

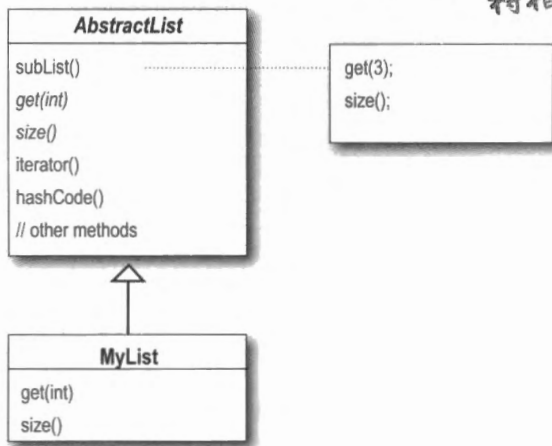
我们必须实现get()和size()方法，两者都被模板方法subList()使用。

我们也实现一个方法set()以便可以修改列表。

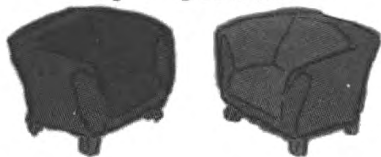
在你的MyStringList实现中测试subList()模板方法，如下：

```
String[] ducks = { "Mallard Duck", "Redhead Duck", "Rubber Duck", "Decoy Duck"};
MyStringList ducksList = new MyStringList(ducks);
List ducksSubList = ducksList.subList(2, 3);
```

创建一个项的子列表，从索引2……
Rubber Duck开始。



炉边谈话



今夜话题：模板方法和策略对比方法。

模板方法：

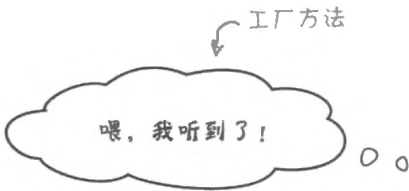
喂，策略，你在我的章节里面做什么？我还以为我得和一些无聊的家伙，像工厂方法，粘在一起呢。

我只是在开玩笑！说正经的，你在这里做什么？我们足足有七章没有你的消息了！

你可能要提醒一下读者你是谁，因为已经这么久不见了。

喂，这听起来很像我在做的事情。但我的意图和你稍有不同。我的工作定义算法的轮廓，但让子类做一些工作。这样，对算法的个别步骤，我可以有不同的实现，但依然控制着算法的结构。好像你不得不放弃对算法的控制。

策略：



是我，不过要小心，你和工厂方法是亲戚，不是吗？

我听说你的章节已经接近尾声，我想过来逛逛，看看情况怎么样。我们有许多共同点，因此我想我也许会有帮助……

我不知道，从第1章开始，我在逛街的时候，老是被拦下来。他们说，“你不是那个什么模式来着……”因此，我想他们知道我是谁。不过为了你，我再说一次吧，我定义一个算法家族，并让这些算法可以互换。因为每个算法都被封装起来，客户可以轻易使用不同的算法。

我不肯定我是不是这样……无论如何，我不是坚持为算法实现使用继承。我通过对象组合给客户提供算法实现的选择。

模板方法：

这我记得。但是我对算法有更多的控制，而且不会有重复代码。事实上，如果我的算法的每一部分都是相同的，除了，比如说，一行，不同，那么我的类要比你的高效得多。我的所有重复代码都放进了超类，所有子类都可以共享。

好吧，我真的替你感到高兴，但是别忘了，环顾四周，我是最常用的模式，为什么？因为我为代码复用提供了一个基础方法，允许子类指定行为。我相信你会看到，这对创建框架是极好的。

怎么会？我的超类是抽象的。

正如我所说的，策略，我真的为你感到高兴。谢谢你来拜访我，但我必须把本章剩下的部分完成。

知道了。不要打电话给我们，我们会打电话给你……

策略：

你可能更高效一点（只是一点点），需要的对象更少。和我的委托模型比起来，你的复杂性也稍小，但我更有弹性，因为我使用对象组合。有了我，客户可以简单地通过使用不同的策略对象，在运行时改变它们的算法。拜托，他们选择把我放在第1章，不是没有原因的！

是的，我猜想……但依赖呢？你的依赖程度比我高。

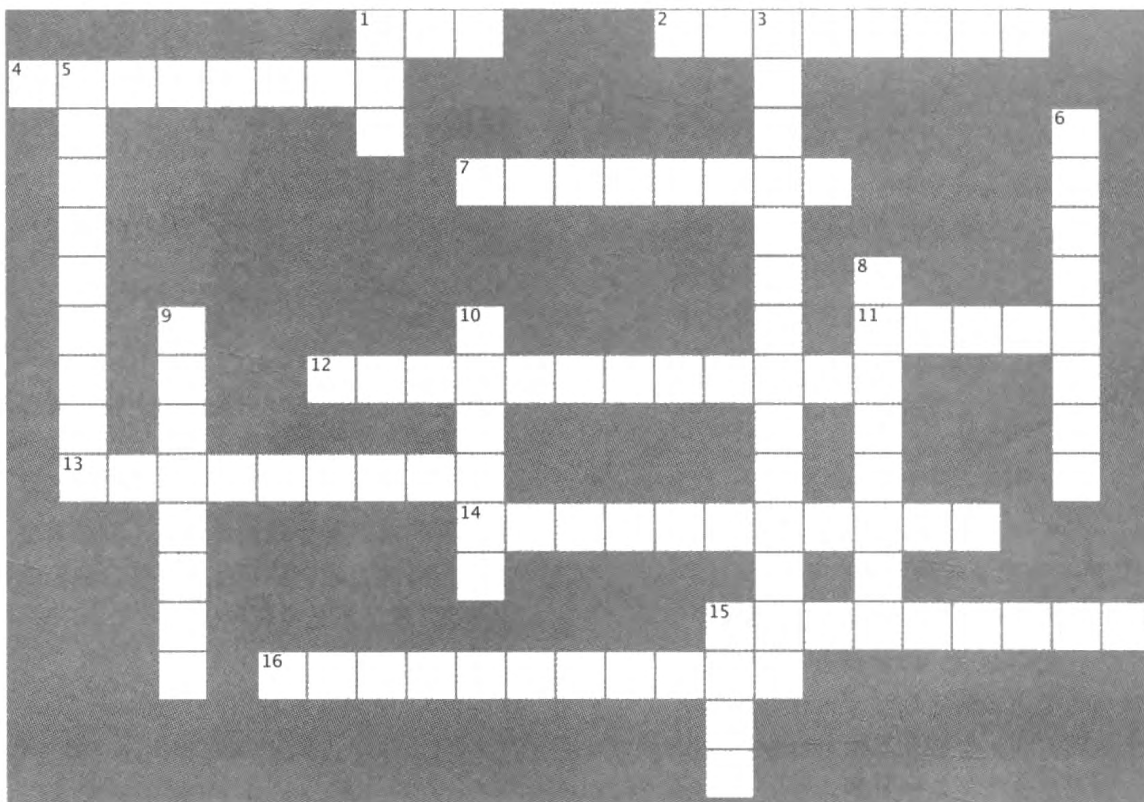
但你不得不依赖于子类中的方法实现，因为这是你算法中的一部分。我不依赖任何人，我自己搞定整个算法！

OK，OK，不要这么敏感。我让你继续工作吧，但是如果你需要我的特殊技能，就让我知道，我总是乐意帮忙的。



设计模式填字游戏

又到这个时候了……



横排

1. Huey、Louie和Dewey都重_____磅。
2. 模板方法通常在_____类中定义。
4. 本章中，我们给你更多_____。
7. 算法中必须由子类供应的步骤通常声明为_____。
11. JFrame 钩子方法，我们覆盖它来打印“I rule!!”
12. _____有一个 subList() 模板方法。
13. 数组中使用的排序类型。
14. 模板方法模式 使用_____to 延迟 实现到其他类。
15. “不要打电话给我，我们会打电话给你”称为_____原则。

竖排

1. 咖啡和_____。
3. 工厂方法是模板方法的一个_____。
5. 模板方法定义一个_____的步骤。
6. 自负的模式。
8. _____算法步骤由钩子 方法实现。
9. 对象村中我们喜爱的咖啡店。
10. 数组类实现其模板方法为一个_____方法。
15. 抽象超类中的一个方法，不做事情或 提供 缺省的 行为，称为_____方法。



你的设计工具箱中的工具

我们把模板方法添加到你的工具箱。有了模板方法，你可以像专家一样复用代码，同时保持对算法的控制。

OO基础

OO原则

封装变化。

组合优于继承。

针对接口编程，而不是针对实现编程。

努力达到交互对象之间的松耦合设计。

类应该对扩展开放，对修改关闭。

依赖于抽象，不依赖于具体类。

只和朋友交谈。

不要打电话给我们，我们会打电话给你。

抽象
封装
多态
继承

我们最新的原则提醒你，超类主导演出，当它们需要的时候，会调用子类的方法，这就跟好莱坞一样。

OO Patterns

还有我们最新的模式，让类实现一个算法，将一些步骤延迟到子类。

模板方法，在一个操作中定义一个算法的骨架，而把一些步骤延迟到子类。模板方法使得子类可以在不改变算法结构的情况下，重新定义算法中的某些步骤。

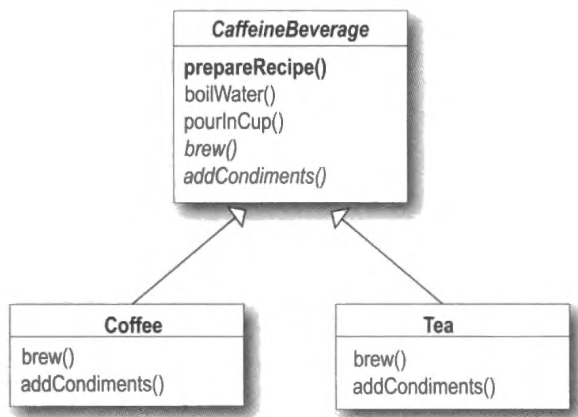
要点

- 模板方法定义了算法的步骤，把这些步骤的实现延迟到子类。
- 模板方法模式为我们提供了一种代码复用的重要技巧。
- 模板方法的抽象类可以定义具体方法、抽象方法和钩子。
- 抽象方法由子类实现。
- 钩子是一种方法，它在抽象类中不做事或者只做缺省的事情，但子类可以覆盖它。
- 为了防止子类改变模板方法中的算法，可以把模板方法声明为final。
- 好莱坞原则告诉我们，把决策权放在高层模块中，以便决定如何以及何时调用低层模块。
- 你会在现实世界代码中看到模板方法模式大量使用，但（任何模式都是）不要期待它们全都是“按书本”设计的。
- 策略模式和模板方法模式都封装算法，前者通过组合，后者通过继承。
- 工厂方法是模板方法的一个特例。



削尖你的铅笔 答案

既然我们已经把prepareRecipe()的实现移进CaffeineBeverage类，那就画一张新的类图：



WHO DOES WHAT? 答案

匹配每个模式及其描述：

模式	描述
模板方法	封装可以互换的行为，并使用委托来决定使用哪个行为。
策略	子类决定如何实现算法中的步骤。
工厂方法	子类决定哪个具体类实例化。



设计模式填字游戏解答

又到这个时刻了……

